Bacharelado em Ciência da Computação
Componente: Computação Gráfica → **Professor:** Claudio Kleina
Aula 2 – Computação gráfica: Linhas e interação com teclado

COMPUTAÇÃO GRÁFICA

1 Criando linhas

Para criar linhas usamos o comando dentro da função desenhar(): **glBegin(GL_LINES)**
Cria uma linha a cada duas coordenadas (x,y) do plano.
Para unir todos os pontos em uma linha, usamos: **GL_LINE_STRIP vs GL_LINE_LOOP**:

- **GL_LINE_STRIP**: Desenha uma linha contínua conectando os pontos A-B-C mas **não** conecta o último ponto de volta ao primeiro. Imagine que você tem uma caneta e pontos A, B, C e D no papel. O GL_LINE_STRIP conecta os pontos na ordem em que aparecem, mas **não volta para o início**.
- **Comportamento:** Conecta A → B, depois B →C, depois C → D.
- **Resultado:** Uma linha "aberta". O último ponto fica "solto".
- **Uso Comum:** Desenhar trajetórias, funções matemáticas (como ondas seno), ou os arcos do coração que fizemos anteriormente.

- **GL_LINE_LOOP**: Fecha automaticamente o desenho conectando o último ponto ao primeiro (C → D)

Para alterar a espessura da linha usamos o comando: **glLineWidth(3.0)** antes de inicializar as linhas. 3.0, neste caso é a espessura da linha.

2 Projeção ortogonal

Permite fazer alteração no plano cartesiano, indicando os valores (x,y) iniciais e finais da tela.
Para alterar a projeção Ortogonal usamos: **gluOrtho2D(0, 500, 0, 500):**
gluOrtho2D(0, 500, 0, 500): Define a área de visualização (projeção ortogonal). Aqui você está dizendo que o canto inferior esquerdo da janela é a coordenada (0,0) e o superior direito é (500, 500). É como definir a escala do seu gráfico. Deve ser inserido **após** a criação da janela (glutCreateWindow).

O que mudou com o gluOrtho2D(0, 500, 0, 500)?

Quando você usa esse comando, a lógica de posicionamento muda completamente. Veja a comparação:

Conceito	OpenGL Padrão (Normalizado)	Seu novo gluOrtho2D
Centro da Tela	(0, 0)	(250, 250)
Canto Inferior Esquerdo	(-1, -1)	(0, 0)
Canto Superior Direito	(1, 1)	(500, 500)
Unidade de Medida	Proporcional (0.5 = metade)	Absoluta (pixels lógicos)

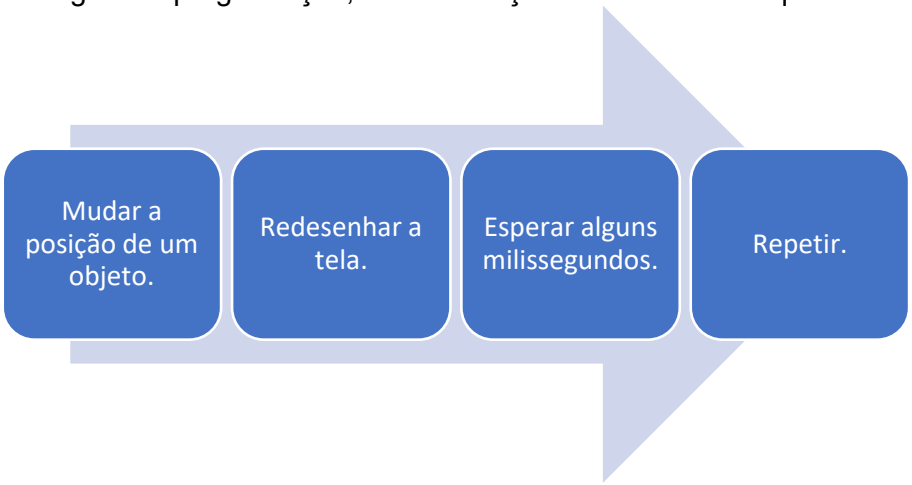
```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
def desenhar():
    glClear(GL_COLOR_BUFFER_BIT) # Limpa a tela (apaga o frame anterior)
    glLineWidth(3.0)  # Largura da linha
    #glBegin(GL_LINES)          # Inicia o modo LINHAS
    #glBegin(GL_LINE_LOOP)  # LINHA ENTRE TODOS OS PONTOS
    #glBegin(GL_LINE_STRIP)  # LINHA ENTRE OS 3 PONTOS (NÃO FECHA)
    glColor3f(1.0, 0.0, 0.0)     # Escolhe a cor Vermelha (R,G,B)
    glVertex2f(0.0, 0.0)        # Posiciona o ponto no centro (0,0)
    glVertex2f(0.5, 0.0)
    glVertex2f(0.5, 0.5)
```

```
    glVertex2f(0.0, 0.5)
    glEnd()                    # Finaliza o desenho
    glFlush()                  # Envia os dados para a placa de vídeo
# Configuração da Janela
glutInit()
glutInitWindowSize(500, 500) #Tamanho da tela em pixels
glutCreateWindow(b"Aula 2 - Linhas")
#gluOrtho2D(0.0, 1.0, 0.0, 1.0)  # define o modo de organização do plano cartesiano
                                 # (esquerda, direita, baixo, cima)
#gluOrtho2D(0, 500, 0, 500) # modo não normalizado
glutDisplayFunc(desenhar)
glutMainLoop()                 # Inicia o programa
```

3 Movimentos com Opengl

Para transformar esse desenho estático em uma animação, precisamos introduzir o conceito de Variáveis de Estado e uma Função de Tempo (Timer).

Na lógica de programação, uma animação nada mais é do que:



Código modificado para fazer um ponto se mover horizontalmente e "rebater" nas bordas da janela:

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

# --- Variáveis de Estado (Onde o ponto está e para onde vai) ---
x_pos = 100.0
direcao = 1.0  # 1 para direita, -1 para esquerda

def myInit():
    glClearColor(1.0, 1.0, 1.0, 1.0)
    glColor3f(0, 0, 0)
    glPointSize(15.0)
    gluOrtho2D(0, 500, 0, 500)

def animacao(valor):
    global x_pos, direcao
    # Lógica de Movimento
    x_pos += (direcao * 5.0) # Incrementa a posição

    # Lógica de Colisão (Se tocar as bordas 0 ou 500, inverte a direção)
    if x_pos >= 500 or x_pos <= 0:
```

```
        direcao *= -1

    glutPostRedisplay() # Avisa ao GLUT que a tela mudou e precisa ser redesenhada
    glutTimerFunc(33, animacao, 0) # Chama esta função novamente em 33ms (~30 FPS)

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    glBegin(GL_POINTS)
    glVertex2f(x_pos, 250) # O X agora é uma variável!
    glEnd()
    glFlush()

# --- Setup padrão ---
glutInit()
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
glutInitWindowSize(500, 500)
glutCreateWindow(b'Ponto em Movimento')
myInit()

glutDisplayFunc(display)
glutTimerFunc(33, animacao, 0) # Inicia o primeiro ciclo do timer
glutMainLoop()
```

O que mudou no "Algoritmo"?
global x_pos: Como a função **animacao** precisa alterar um valor que está fora dela, usamos o global. Isso é um ótimo gancho para explicar Escopo de Variáveis em sala de aula.
glutTimerFunc(ms, função, valor): Este é o "coração" da animação. Ele diz ao sistema: "Espere 33* milissegundos e execute a função animacao novamente".
*33ms é o tempo aproximado para rodar a 30 quadros por segundo (FPS).

glutPostRedisplay(): Sem esse comando, o computador mudaria o valor de x_pos na memória, mas você não veria nada na tela. Ele **força a chamada da função display().**

A **Condicional if:** Aqui aplicamos a lógica pura. Se o ponto atingir o limite do nosso **gluOrtho2D (0 ou 500)**, multiplicamos a direção por -1 (o que inverte o sinal do movimento).

Desafios:
1) Crie um novo ponto para se mover na vertical (eixo Y) ao mesmo tempo. Será necessário criar uma **y_pos** e uma **direcao_y**.
2) Toda a vez que o ponto tocar na parede, o ponto deve mudar de cor.

Exemplo de código usando a matriz de cores:

```
cores = [
    (0.0, 0.0, 0.0), # Preto
    (1.0, 0.0, 0.0), # Vermelho
    (0.0, 0.5, 0.0), # Verde
    (0.0, 0.0, 1.0), # Azul
    (1.0, 0.5, 0.0)  # Laranja
]
```

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
```

```
import random # Vamos usar para iniciar com uma cor aleatória

# --- Estado do Sistema ---
x_pos = 250.0
direcao = 1.0
indice_cor = 0

# --- Lista de Cores (Paleta RGB) ---
# Cada item é uma tupla (R, G, B)
cores = [
    (0.0, 0.0, 0.0), # Preto
    (1.0, 0.0, 0.0), # Vermelho
    (0.0, 0.5, 0.0), # Verde
    (0.0, 0.0, 1.0), # Azul
    (1.0, 0.5, 0.0)  # Laranja
]

def myInit():
    glClearColor(1.0, 1.0, 1.0, 1.0)
    glPointSize(20.0) # Aumentei para destacar a cor
    gluOrtho2D(0, 500, 0, 500)

def animacao(valor):
    global x_pos, direcao, indice_cor

    x_pos += (direcao * 7.0)

    # Lógica de Colisão + Troca de Cor
    if x_pos >= 500 or x_pos <= 0:
        direcao *= -1
        # Lógica de Índice Circular: vai de 0 a 4 e volta para 0
        indice_cor = (indice_cor + 1) % len(cores)

    glutPostRedisplay()
    glutTimerFunc(33, animacao, 0)

def display():
    glClear(GL_COLOR_BUFFER_BIT)

    # Define a cor atual baseada no índice da nossa lista
    r, g, b = cores[indice_cor]
    glColor3f(r, g, b)

    glBegin(GL_POINTS)
    glVertex2f(x_pos, 250)
    glEnd()
    glFlush()

# --- Setup ---
glutInit()
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
glutInitWindowSize(500, 500)
glutCreateWindow(b'Ponto Colorido Rebatendo')
myInit()
glutDisplayFunc(display)
```

```
glutTimerFunc(33, animacao, 0)
glutMainLoop()
```

O uso de uma **Estrutura de Dados (Lista** de cores) armazena múltiplos valores. Dessa forma, não precisamos de 5 variáveis diferentes para 5 cores; uma lista resolve.
O **Operador Módulo (%):** A linha **indice_cor = (indice_cor + 1) % len(cores)** é um truque clássico de programação. Ela garante que o índice nunca ultrapasse o tamanho da lista, criando um loop infinito (0, 1, 2, 3, 4, 0, 1...).

Desempacotamento de Tuplas: A linha **r, g, b = cores[indice_cor]** permite extrair múltiplos valores de um único item da lista de forma elegante no Python.

Faça o teste: Adicione uma nova cor à lista cores (ex: (0.5, 0.0, 0.5) para roxo) e analise o resultado.

3 Controle de movimentos pelo teclado
Para interagir através do teclado, usaremos a função **glutSpecialFunc**, que é específica para capturar teclas que não são letras (como as setas, F1, F2, etc.).

Segue o código transformado em um sistema interativo:

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

# --- Estado do Sistema (Posição do Jogador) ---
x_pos = 250.0
y_pos = 250.0
velocidade = 15.0 # Quantos pixels ele pula por clique

def myInit():
    glClearColor(1.0, 1.0, 1.0, 1.0)
    glColor3f(0.2, 0.4, 0.8) # Um azul bonito para o nosso "personagem"
    glPointSize(25.0)        # Ponto grande para facilitar a visão
    gluOrtho2D(0, 500, 0, 500)

def teclas_especiais(tecla, x, y):
    global x_pos, y_pos

    # Lógica de movimentação com as setas
    if tecla == GLUT_KEY_UP:
        y_pos += velocidade
    elif tecla == GLUT_KEY_DOWN:
        y_pos -= velocidade
    elif tecla == GLUT_KEY_LEFT:
        x_pos -= velocidade
    elif tecla == GLUT_KEY_RIGHT:
        x_pos += velocidade

    # Tratamento de Bordas (Não deixa o ponto sair da tela)
    x_pos = max(0, min(500, x_pos))
    y_pos = max(0, min(500, y_pos))

    glutPostRedisplay() # Redesenha a tela após o movimento
```

```
def display():
    glClear(GL_COLOR_BUFFER_BIT)
    glBegin(GL_POINTS)
    glVertex2f(x_pos, y_pos) # Agora x e y são controlados pelo teclado
    glEnd()
    glFlush()

# --- Setup ---
glutInit()
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
glutInitWindowSize(500, 500)
glutCreateWindow(b'Controle o Ponto com as Setas')
myInit()

glutDisplayFunc(display)
glutSpecialFunc(teclas_especiais) # Registra a função de captura das setas
glutMainLoop()
```

No programa:
→**Event-Driven Programming (Programação Orientada a Eventos**): O programa não está mais rodando sozinho. Ele fica "dormindo" no glutMainLoop e só acorda quando um Evento de Teclado acontece. Isso é a base de quase todos os softwares modernos.
→**Tratamento de Limites (Clamping):** As linhas **max(0, min(500, x_pos))** são uma forma elegante de lógica para impedir que o personagem "fuja" da janela.
→**Cada tecla tem um código** (ex: **GLUT_KEY_UP**) que o sistema operacional traduz em uma ação no seu algoritmo.

4 Colisão
A colisão ocorre quando as posições de dois objetos coincidem (ou estão muito próximas).
Segue um exemplo para analisarmos:

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import math # Importação necessária para usar a função de raiz quadrada (sqrt)

# --- Estado do Sistema ---
x_pos, y_pos = 50.0, 50.0   # Coordenadas atuais do jogador
x_obj, y_obj = 400.0, 400.0 # Coordenadas fixas do objetivo
velocidade = 20.0
ganhou = False

def myInit():
    glClearColor(1.0, 1.0, 1.0, 1.0)
    glPointSize(25.0)
    gluOrtho2D(0, 500, 0, 500)

def teclas_especiais(tecla, x, y):
    global x_pos, y_pos, ganhou

    if ganhou: return

    if tecla == GLUT_KEY_UP:    y_pos += velocidade
    elif tecla == GLUT_KEY_DOWN:  y_pos -= velocidade
```

```
    elif tecla == GLUT_KEY_LEFT:  x_pos -= velocidade
    elif tecla == GLUT_KEY_RIGHT: x_pos += velocidade

    # --- INÍCIO DA LÓGICA DE COLISÃO ---

    # 1. Aplica o Teorema de Pitágoras: d = sqrt( (x2-x1)^2 + (y2-y1)^2 )
    # Esta linha calcula a distância em linha reta entre o centro do jogador e o centro do objetivo.
    distancia = math.sqrt((x_pos - x_obj)**2 + (y_pos - y_obj)**2)

    # 2. Verifica se a distância calculada é menor que a soma dos raios dos pontos (limiar de
colisão).
    # Como os pontos têm tamanho 25.0, uma distância menor que 30.0 garante que eles
visualmente se toquem.
    if distancia < 30.0:
        ganhou = True # Altera o estado do sistema para indicar vitória
        print("SISTEMA: OBJETIVO ALCANÇADO! VOCÊ CHEGOU!") # Feedback no console

    # --- FIM DA LÓGICA DE COLISÃO ---

    glutPostRedisplay()

def display():
    glClear(GL_COLOR_BUFFER_BIT)

    # Desenha o Objetivo (Vermelho)
    glColor3f(1.0, 0.0, 0.0)
    glBegin(GL_POINTS)
    glVertex2f(x_obj, y_obj)
    glEnd()

    # Desenha o Jogador (Azul ou Verde se ganhar)
    if ganhou:
        glColor3f(0.0, 0.8, 0.0)
    else:
        glColor3f(0.2, 0.4, 0.8)

    glBegin(GL_POINTS)
    glVertex2f(x_pos, y_pos)
    glEnd()

    glFlush()

# --- Setup ---
glutInit()
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
glutInitWindowSize(500, 500)
glutCreateWindow(b'Desafio: Alcance o Objetivo')
myInit()
glutDisplayFunc(display)
glutSpecialFunc(teclas_especiais)
glutMainLoop()
```

O que esse código ensina sobre Matemática e Computação?

1. **Distância Euclidiana:** aplicação prática do Teorema de Pitágoras para detectar se dois objetos se tocaram no plano 2D.
2. **Variáveis de Controle (Flags):** A variável ganhou funciona como uma chave (liga/desliga). Se for True, o sistema para de aceitar comandos de movimento, demonstrando como estados lógicos controlam o fluxo do programa.
3. **Feedback Visual:** A mudança de cor de Azul para Verde no if ganhou: ensina que o estado interno do software deve ser refletido na interface para o usuário.

ATIVIDADES

1) Modifique o comando gluOrtho2D. Altere os limites para (0, 1000, 0, 1000). O que acontece com a velocidade do ponto e o seu tamanho visual na tela?

2) Altere o código para que, além de mudar para verde ao colidir, o jogador mude para uma cor aleatória toda vez que uma tecla de seta for pressionada.

3) Modifique a função display para que o comando glClear não seja chamado. Movimente o ponto. Analise por que o rastro aparece e como o Color Buffer funciona.

4) Implemente o suporte para as teclas de letras 'W', 'A', 'S', 'D' usando **glutKeyboardFunc**. Permita que, ao pressionar 'W' e 'D' simultaneamente (ou em sequência rápida), o ponto se mova diagonalmente. **(* Ver código ao final das questões)**

5) implemente o Teletransporte (Wrap-around)→ Em vez de usar o Clamping (travar nas bordas), faça com que o ponto, ao sair pelo lado direito da tela (x > 500), reapareça no lado esquerdo (x < 0), e vice-versa.

6) Faça com que o ponto vermelho (objetivo) não fique parado. Ele deve se mover aleatoriamente (usando a biblioteca random) a cada 2 segundos.

Nível avançado!!!!

7) Crie uma lista de 5 objetivos em posições aleatórias. O jogador deve "coletar" todos. Quando um objetivo é coletado, ele deve sumir da tela. O jogo só termina quando a lista estiver vazia.

8) Obstáculo Repulsor: Adicione um terceiro ponto (cor amarela) que atua como um "ímã invertido". Se o jogador chegar a menos de 50 pixels dele, o jogador deve ser "empurrado" na direção oposta automaticamente.

9) Implemente um sistema de pontuação e tempo: Exiba no console o tempo que o usuário levou para atingir o objetivo.

10) Adicione um limite de 10 segundos. Se o usuário não chegar ao ponto vermelho, o ponto azul deve ficar cinza e o jogo deve travar (Game Over).

Bônus: Adicione uma função de "Reset" pressionando a tecla 'R'.

*** Para verificar teclas comuns:**

```
def teclado_comum(tecla, x, y):
```

```
    """Trata as teclas WASD (letras)."""
    global x_pos, y_pos
    if ganhou: return

    # O .lower() serve para aceitar tanto 'W' quanto 'w'
    tecla = tecla.lower()

    if tecla == b'w':   y_pos += velocidade
    elif tecla == b's': y_pos -= velocidade
    elif tecla == b'a': x_pos -= velocidade
    elif tecla == b'd': x_pos += velocidade

    # Tecla ESC para sair
    if tecla == chr(27).encode():  # ou  b'\x1b' em hexadecimal
        glutLeaveMainLoop()

    verificar_colisao()
    glutPostRedisplay()

def teclado_especial(tecla, x, y):
    """Trata as Setas do Teclado."""
    global x_pos, y_pos
    if ganhou: return

    if tecla == GLUT_KEY_UP:    y_pos += velocidade
    elif tecla == GLUT_KEY_DOWN:  y_pos -= velocidade
    elif tecla == GLUT_KEY_LEFT:  x_pos -= velocidade
    elif tecla == GLUT_KEY_RIGHT: x_pos += velocidade

    verificar_colisao()
    glutPostRedisplay()


# --- Setup Final ---
glutInit()
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
glutInitWindowSize(500, 500)
glutCreateWindow(b'Desafio: Setas ou WASD')

myInit()

glutDisplayFunc(display)
# REGISTRO DAS DUAS FUNÇÕES DE TECLADO:
glutKeyboardFunc(teclado_comum)   # Para letras (WASD)
glutSpecialFunc(teclado_especial) # Para setas

print("Controles Ativos: SETAS ou WASD. Pressione ESC para sair.")
glutMainLoop()
```

Bom estudo!